

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – REGEX PATTERN MATCHING ACTIVITY

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO3	Assessment:	Tool 1 – Regex Pattern Matching Activity
Weightage:		Total Marks:	25 Marks
CO3 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex. (BL3 and BL4).			
Student Name:	Rakshitha V B	Reg. No.:	192324004
Section / Batch:	2023 - 2027	Date of Submission:	14-08-2026
Faculty In-charge:	K. Veena devi	Project Mode:	Individual Submission

Code of Conduct

I Rakshitha V B (192324004) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Understanding (5 Marks)	Pattern Design (5 Marks)	Data Extraction and Validation (5 Marks)	Explanation (5 Marks)	Applications (5 Marks)	Total (25 Marks)

Set - 01

1. Break down the regex `\d{3}-\d{2}-\d{4}` token by token. What does each component match, and what real-world data format does this pattern describe?
Consider quantifiers, character classes, and literal characters separately.
2. Given the pattern `^(https?|ftp):\/\/[^\s]+`, identify which part handles protocol variation and explain how the alternation operator changes the set of matching strings.
Trace how `https?` differs from `https|http` in coverage.
3. Compare greedy vs. lazy quantifiers using `<.*>` versus `<.*?>` on the string `<b>bold</b>`.
Trace exactly what each engine captures and explain why the outputs differ.
Draw the backtracking steps for greedy matching.
4. Analyze why the pattern `(a+)+` causes catastrophic backtracking on an input like `aaaaaX`.
How does the nested quantifier structure create exponential match attempts?
Count the distinct ways the engine can partition the 'a' characters between the outer and inner groups.
5. Write a regex to extract all email addresses from an unstructured customer feedback log.
What assumptions does your pattern make, and which valid email formats might it miss?
Think about subdomains, plus-addressing, and unusual TLDs.
6. A dataset contains dates in mixed formats: `MM/DD/YYYY`, `DD-Mon-YYYY`, and `YYYY.MM.DD`.
Design a single regex with named capture groups that normalises all three formats into a structured output.
Use named groups like `(?P<year>...)` in Python.
7. You are preprocessing tweets for sentiment analysis. Write a pipeline of regex substitutions that removes URLs, mentions, hashtag symbols (but keeps the word), and repeating punctuation. Justify the order of operations.
Order matters — removing URLs before mentions avoids accidental partial matches.
8. Using lookaheads and lookbehinds, write a pattern that matches prices in a product catalog only when they appear inside a `<price>` tag and exceed \$99, without consuming the XML tags in the match.
Combine a lookbehind for the opening tag with a lookahead for the closing tag.
9. A colleague proposes using `.*` as the default pattern when the exact format is unknown. Critique this approach in the context of query processing — what correctness, performance, and maintainability risks does it introduce?
Think about false positives in SQL LIKE queries vs. full regex engines.
10. Compare the use of regex versus purpose-built parsers (e.g. HTML parsers, CSV readers) for structured data extraction. In what scenarios does regex become the wrong tool, and what signals in the data should prompt you to switch?
Consider recursive structures, escaped delimiters, and nested tags.

11. You are given two patterns to validate IPv4 addresses: `\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}` and a more complex range-validating pattern. Evaluate the trade-off between regex simplicity and semantic correctness — when is 'good enough' acceptable in a data pipeline? Does the simpler pattern accept `999.999.999.999`? Does that matter in your pipeline context?

12. Assess the suitability of regex-based tokenisation versus modern NLP tokenisers (e.g. byte-pair encoding) for preprocessing a multilingual query corpus. Under what linguistic conditions does regex fail systematically?

Consider compound words in German, agglutinative languages, and Unicode character classes.

13. Design a regex-based data validation layer for a healthcare query system that must parse ICD-10 codes, phone numbers, and patient IDs. How would you organise these patterns for maintainability, and how does your design change if the system needs to run at 10,000 queries per second?

Think about precompiled patterns, DFA vs NFA engines, and separation of validation logic.

14. Build a framework for automatically generating regex patterns from a set of positive and negative example strings. What algorithmic challenges arise, and how would you evaluate the precision and recall of the generated patterns on held-out test data?

This connects to the field of 'regex synthesis' — think about over-fitting to examples.

15. You are designing a query rewriting engine that uses regex to detect and transform SQL injection patterns in user inputs before they reach the database. Synthesise a strategy that balances security coverage, false positive rate, and query latency. What are the limits of a regex-only approach?

Consider parameterised queries as the ground truth — where does regex complement rather than replace them?

Q1. Break down the regex `\d{3}-\d{2}-\d{4}` token by token.

Token-by-Token Breakdown

Token	Meaning
<code>\d{3}</code>	Matches exactly 3 decimal digits (0–9)
<code>-</code>	Literal hyphen character
<code>\d{2}</code>	Matches exactly 2 decimal digits (0–9)
<code>-</code>	Literal hyphen character
<code>\d{4}</code>	Matches exactly 4 decimal digits (0–9)

What Real-World Format Does This Describe?

This regex describes a US Social Security Number (SSN) in the standard format:

Example match: `123-45-6789`

Components:

- `\d` — character class shorthand for `[0-9]` (any digit)
- `{n}` — exact quantifier: match precisely `n` occurrences
- Hyphens — literal separator characters, not escaped because they are not inside `[ ]`

Possible real-world formats: SSN (123-45-6789), US phone area patterns, product serial codes.

Q2. Identify which part of `^(https?|ftp):\/\/[^\s]+$` handles protocol variation.

Pattern Breakdown

Part	Purpose
<code>^</code>	Anchor — match must start at beginning of string
<code>(https? ftp)</code>	Alternation group — matches 'http', 'https', or 'ftp'
<code>https?</code>	'http' with optional 's' (? = zero or one of preceding char)
<code> ftp</code>	OR: also match the literal string 'ftp'
<code>:\/\/</code>	Literal <code>://</code> (slashes escaped)
<code>[^\s]+</code>	One or more non-whitespace characters (the URL path)
<code>\$</code>	Anchor — match must end at end of string

https? vs https|http — Coverage Difference

- `https?` → matches 'http' OR 'https' (the 's' is optional via ?)
- `https|http` → matches 'https' OR 'http' (two full alternates)
- Difference: `https?` is more concise; both produce same string coverage for this case
- However, `https?` also accepts partial strings like 'http' + 's' in other contexts (e.g. inside a word)
- Anchors `^` and `$` ensure the protocol group must match the whole string start, so both behave equivalently here

Q3. Compare greedy `<.*>` vs. lazy `<.*?>` on the string `<b>bold</b>`.

Greedy: `<.*>`

Input: `<b>bold</b>`
Pattern: `<.*>`
Match: `<b>bold</b>` (entire string — one match)

Backtracking steps (greedy):

- `.*` consumes as much as possible: `'b>bold</b>'`
- Engine tries to match `'>'` — reaches end of string, backtracks
- Backtracks one char at a time until `'>'` found at last `'>'`
- Final match: `<b>bold</b>` — the ENTIRE content between outermost `<` and `>`

Lazy: `<.*?>`

Input: `<b>bold</b>`
Pattern: `<.*?>`
Match 1: `<b>` Match 2: `</b>`

Lazy quantifier stops at the FIRST `'>'` it finds:

- ▶ `. * ?` matches as few characters as possible
- ▶ After `<`, matches `'b'` then immediately tries `'>'` — succeeds → match = `<b>`
- ▶ Engine continues and finds second match: `</b>`

Key Difference

Greedy = expand as far as possible, then backtrack. Lazy = stop as soon as possible.

Q4. Analyze why `(a+)+` causes catastrophic backtracking on input like `aaaaaX`.

Why Catastrophic Backtracking Occurs

The pattern `(a+)+` has a nested quantifier: the inner `a+` and the outer `+` both try to claim 'a' characters.

Exponential Partitioning

For `n='a'` characters, the engine can partition them between inner/outer groups in $2^{(n-1)}$ ways:

Input: `aaaaa` (5 a's)

Partitions: `(aaaaa)`, `(aaaa)(a)`, `(aaa)(aa)`, `(aaa)(a)(a)`, `(aa)(aaa)`, ...

For 5 a's → 16 distinct partitions. For 10 a's → 512 partitions. For `n` a's → $2^{(n-1)}$ attempts.

- ▶ Every partition is tried when the 'X' causes the overall match to fail
- ▶ After each failure, the engine backtracks and tries a different inner/outer partition
- ▶ The 'X' at the end ensures the match always fails, forcing ALL partitions to be exhausted

How to Fix

- ▶ Use atomic groups: `(?>a+)` — prevents backtracking into the group
- ▶ Rewrite as `a+` — equivalent match, no nesting
- ▶ Use possessive quantifiers: `(a++)` in engines that support them

Q5. Write a regex to extract all email addresses from an unstructured customer feedback log.

Regex Pattern

`[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}`

Component Explanation

Part	Matches
<code>[a-zA-Z0-9._%+\-]+</code>	Local part: letters, digits, dots, underscores, %, +, hyphen (1 or more)
<code>@</code>	Literal @ symbol
<code>[a-zA-Z0-9.\-]+</code>	Domain: letters, digits, dots, hyphens (1 or more)
<code>\.</code>	Literal dot before TLD
<code>[a-zA-Z]{2,}</code>	TLD: 2 or more letters (e.g. com, org, co, io, museum)

Assumptions and Limitations

- ▶ Assumes standard ASCII emails — misses Unicode/IDN email addresses
- ▶ Does NOT validate length constraints (local part max 64 chars, domain max 255 chars)
- ▶ May miss plus-addressing if '+' not in character class (included above)
- ▶ May miss unusual TLDs over 10 chars though `{2,}` handles most
- ▶ Does not handle quoted local parts: e.g. `"john doe"@example.com`
- ▶ Subdomains (`user@mail.example.co.uk`) — handled correctly by `[a-zA-Z0-9.\-]+`

Q6. Design a single regex with named capture groups to normalise MM/DD/YYYY, DD-Mon-YYYY, and YYYY.MM.DD.

Python Regex with Named Groups

```
import re
pattern = r'''
    (?P<m1>\d{2}) / (?P<d1>\d{2}) / (?P<y1>\d{4})           # MM/DD/YYYY
    | (?P<d2>\d{2}) - (?P<mon>[A-Za-z]{3}) - (?P<y2>\d{4}) # DD-Mon-YYYY
    | (?P<y3>\d{4}) \. (?P<m3>\d{2}) \. (?P<d3>\d{2})      # YYYY.MM.DD
'''
rx = re.compile(pattern, re.VERBOSE)
```

Normalisation Logic After Match

```
month_map = {'Jan': '01', 'Feb': '02', 'Mar': '03', 'Apr': '04',
             'May': '05', 'Jun': '06', 'Jul': '07', 'Aug': '08',
             'Sep': '09', 'Oct': '10', 'Nov': '11', 'Dec': '12'}

for m in rx.finditer(text):
    if m.group('y1'): # MM/DD/YYYY
        year, month, day = m.group('y1'), m.group('m1'), m.group('d1')
    elif m.group('y2'): # DD-Mon-YYYY
        year = m.group('y2')
        month = month_map[m.group('mon')].capitalize()
        day = m.group('d2')
    else: # YYYY.MM.DD
        year, month, day = m.group('y3'), m.group('m3'), m.group('d3')
    print(f'{year}-{month}-{day}') # ISO 8601 output
```

Named Groups Used

- y1/y2/y3, m1/m3, d1/d2/d3, mon — each group uniquely named per format
- re.VERBOSE allows multi-line pattern with inline comments
- Output format: YYYY-MM-DD (ISO 8601 — universal standard)

Q7. Write a regex pipeline to preprocess tweets: remove URLs, mentions, hashtag symbols, repeating punctuation.

Pipeline Steps (Order Matters)

Step	Regex Substitution
1. Remove URLs	<code>re.sub(r'https?://\S+ www.\S+', '', text)</code>
2. Remove @mentions	<code>re.sub(r'@\w+', '', text)</code>
3. Remove # but keep word	<code>re.sub(r'#(\w+)', r'\1', text)</code>
4. Remove repeating punct	<code>re.sub(r'([^\w\s])\1+', r'\1', text)</code>
5. Strip extra whitespace	<code>re.sub(r'\s+', ' ', text).strip()</code>

Why This Order?

- Step 1 first — URLs like 'https://t.co/@user' would be partially matched by @mention pattern if not removed first
- Step 2 before Step 3 — '@#trending' would confuse hashtag removal if mentions not removed first
- Step 3 keeps the word — '#Python' becomes 'Python' not '' (using capture group \1)
- Step 4 last content step — normalises '!!!' to '!', '...' to '.' for cleaner sentiment features
- Step 5 final — removes double spaces left by earlier removals

Example

```
Input: 'Love @user this! Check https://t.co/abc #AI is great!!!'
Output: 'Love this! Check AI is great!'
```

Q8. Using lookaheads and lookbehinds, match prices inside <price> tag exceeding \$99.

Regex Pattern

```
(?<=<price>\$) (?:(100|[1-9]\d{2,}|\d{4,})) (?=</price>)
```

For Decimal Prices

```
(?<=<price>\$) (\d{3,} (?:(?:\.\d{2})?)) (?=</price>)
```

Explanation

Component	Purpose
(?<=<price>\\$)	Lookbehind — asserts '<price>\$' comes before match (not consumed)
(\d{3,}...)	Matches integers with 3+ digits (100 and above = >\$99)
(?:\.\d{2})?	Optional decimal part e.g. .99 (non-capturing group)
(?=</price>)	Lookahead — asserts '</price>' comes after match (not consumed)

Why Lookahead/Lookbehind?

- ▶ Lookarounds are zero-width assertions — they check context but do NOT include XML tags in the match result
- ▶ Only the price number is captured: '149.99' not '<price>\$149.99</price>'
- ▶ This allows direct use of the matched string as a number without stripping tags

Example

```
Input: <price>$149.99</price> <price>$45.00</price>
Match: 149.99      (45.00 excluded — does not exceed $99)
```

Q9. Critique using .* as the default pattern when the exact format is unknown.

What .* Does

.* matches any character (except newline by default) zero or more times — essentially matches everything.

Risks in Query Processing

Risk Category	Description
Correctness — False Positives	.* will match invalid data alongside valid data — no semantic filtering occurs
Correctness — Semantic Blindness	Matches '999abc!!!' as happily as '123-45-6789' — no structure enforced
Performance — Greedy Backtracking	In NFA engines, .* causes excessive backtracking in complex patterns
Performance — SQL LIKE Equivalent	LIKE '%...%' scans full table — no index use; similar cost in regex on large corpora
Maintainability — Intent Loss	Future developers cannot determine what format was intended from the pattern
Maintainability — Silent Failures	Data quality issues pass undetected — corrupt rows are accepted silently

Better Alternatives

- ▶ Use specific character classes: [A-Za-z0-9]+ instead of .+
- ▶ Use anchors (^ and \$) to restrict scope
- ▶ Use .+ only as a last resort with additional post-match validation
- ▶ In SQL: use LIKE 'prefix%' rather than LIKE '%anything%' to enable index use

Q10. Compare regex versus purpose-built parsers for structured data extraction.

Regex vs Purpose-Built Parsers

Criterion	Regex
HTML Parsing	Fails on nested/malformed tags
CSV with escaping	Breaks on quoted commas: 'a, "b,c", d'
JSON/XML	Cannot handle recursive nesting
Speed (simple)	Faster for simple flat patterns
Portability	Works cross-language
Maintainability	Brittle for complex formats

When to Switch from Regex to a Parser

- ▶ Recursive / nested structures: HTML div inside div, JSON arrays inside objects
- ▶ Escaped delimiters: CSV fields containing commas, XML entities (& and <)
- ▶ Context-sensitive rules: Markdown where * means different things depending on context
- ▶ Data with Unicode: multi-byte chars, right-to-left scripts, combining characters

Signals in Data That Suggest Switching

- ▶ Your regex starts using 5+ levels of nesting or alternation
- ▶ You are manually unescaping after matching
- ▶ You find edge cases breaking the pattern faster than you can fix it

Q11. Evaluate trade-off between `\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}` and a range-validating IPv4 pattern.

Simple Pattern

```
\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}
```

- ▶ Matches: 192.168.1.1 ✓
- ▶ Also matches: 999.999.999.999 ✓ (INVALID IP — no range check)
- ▶ Also matches: 0.0.0.0 and 255.255.255.255 ✓

Semantically Correct Pattern (Range-Validating)

```
(25[0-5]|2[0-4]\d|1\d{2}|[1-9]?)\d\.(25[0-5]|2[0-4]\d|1\d{2}|[1-9]?)\d\.(25[0-5]|2[0-4]\d|1\d{2}|[1-9]?)\d\.(25[0-5]|2[0-4]\d|1\d{2}|[1-9]?)\d
```

- ▶ Matches only 0–255 in each octet → no false positives

Trade-off Evaluation

Criterion	Simple Pattern
Simplicity	Easy to write and read
Correctness	False positives (999.x.x.x accepted)
Performance	Faster (fewer alternations)
'Good Enough'?	Yes — if downstream systems validate IPs

Verdict: Use the simple pattern for quick log scanning; use the range-validating pattern for security-sensitive pipelines.

Q12. Assess regex-based tokenisation versus modern NLP tokenisers for multilingual query corpus.

Regex Tokenisation

```
Simple split: re.split(r'\s+', text)
Word tokens: re.findall(r'\b\w+\b', text)
```

When Regex Fails Systematically

Linguistic Condition	Why Regex Fails
German compound words	'Donaudampfschiffahrt' is one word; splitting on spaces gives wrong tokens
Agglutinative languages	Turkish/Finnish pack many morphemes into one token; regex can't split morphemes
Chinese/Japanese	No spaces between words; \b word boundary is meaningless
Arabic script	Right-to-left, joining letters change shape; Unicode categories needed
Contractions (English)	"don't" → ['don', 't'] requires language-aware splitting
Emoji/symbols	Multi-codepoint emoji: regex needs \X (extended grapheme cluster)

BPE (Byte-Pair Encoding) Advantages

- ▶ Language-agnostic — learns subword units from corpus statistics
- ▶ Handles unknown/rare words by breaking into known subword units
- ▶ Used in BERT, GPT — state-of-the-art performance on multilingual tasks

When Regex Is Still Useful

- ▶ Pre-cleaning (removing HTML, URLs, control chars) before feeding to NLP tokeniser
- ▶ Domain-specific tokens: ICD codes, model numbers, product SKUs with fixed format

Q13. Design a regex-based data validation layer for a healthcare query system (ICD-10, phone, patient ID).

Compiled Pattern Definitions

```
import re
PATTERNS = {
    'icd10': re.compile(r'^[A-TV-Z][0-9][0-9A-Z](\.[0-9A-Z]{1,4})?$'), re.I),
    'phone_in': re.compile(r'^(\+91)?[6-9]\d{9}$'), # Indian mobile
    'phone_us': re.compile(r'^\+1[2-9]\d{2}[2-9]\d{6}$'), # US NANP
    'patient_id': re.compile(r'^PAT-\d{6}-[A-Z]{2}$'), # e.g. PAT-192324-RB
}
```

Validation Function

```
def validate(field_type, value):
    pattern = PATTERNS.get(field_type)
    if pattern is None: raise ValueError(f'Unknown field: {field_type}')
    return bool(pattern.fullmatch(value))
```

Design for 10,000 Queries/Second

- ▶ Pre-compile all patterns at startup (re.compile) — avoids recompilation cost per query
- ▶ Use re.fullmatch not re.search — eliminates partial match false positives, also faster
- ▶ Cache compiled patterns in a dict — O(1) lookup per field type
- ▶ DFA-based engine (e.g. Google RE2) preferred over NFA (Python re) at high throughput — RE2 guarantees O(n) time
- ▶ Separate validation from query logic — validate at API gateway before DB access
- ▶ Use connection pooling — validation is CPU-bound; keep separate thread pool from DB I/O threads

Q14. Build a framework for automatically generating regex patterns from positive and negative examples.

Framework Overview

Stage	Description
1. Tokenise examples	Break positive strings into character-class tokens (digit, alpha, punct, etc.)
2. Find common structure	Align tokens across examples — find shared prefix, suffix, separators
3. Generalise wildcards	Replace fixed chars with character classes; infer quantifiers from length ranges
4. Prune with negatives	Reject generated patterns that match any negative example
5. Evaluate P/R	Test on held-out positive and negative sets — measure precision and recall

Algorithmic Challenges

- ▶ Overfitting: Pattern exactly matches only seen examples (e.g. `^(abc|def|ghi)$`) — no generalisation
- ▶ Underfitting: Pattern too general (e.g. `.*`) — matches everything including negatives
- ▶ Alignment ambiguity: `'jan-2024'` and `'01/2024'` — hard to align tokens automatically
- ▶ Quantifier inference: How many is 'some'? Requires min/max length statistics from examples
- ▶ Regex synthesis is NP-hard in general — heuristics are needed for practical use

Evaluation Metrics

Precision = $TP / (TP + FP)$ — of strings matched, how many are truly valid?

Recall = $TP / (TP + FN)$ — of valid strings, how many does pattern match?

- ▶ Use held-out test set (not seen during generation) — prevents overfitting to examples
- ▶ Real tools: FlashFill (Microsoft), Automata Learning (RPNI algorithm), RegexLearn

Q15. Design a query rewriting engine using regex to detect and transform SQL injection patterns.

Regex-Based Detection Patterns

```
SQLI_PATTERNS = [  
    re.compile(r"'\\s*OR\\s*'?1'?\\s*=\\s*'?1'", re.I),      # OR 1=1  
    re.compile(r"--\\s*$", re.M),                          # comment injection  
    re.compile(r";\\s*DROP\\s+TABLE", re.I),                # stacked DROP  
    re.compile(r"UNION\\s+(ALL\\s+)?SELECT", re.I),          # UNION SELECT  
    re.compile(r"'\\s*;\\s*", re.I),                        # statement terminator  
]  
  
def is_sqli(user_input):  
    return any(p.search(user_input) for p in SQLI_PATTERNS)
```

Strategy: Layered Defense

Layer	Mechanism
1. Parameterised Queries	Ground truth defense — ALWAYS use; regex complements, never replaces
2. Regex Pre-filter	Reject or flag obvious patterns before hitting DB — reduces attack surface
3. Input Whitelist	Allow only expected chars per field: <code>[A-Za-z0-9]</code> for names, <code>\\d+</code> for IDs
4. WAF Integration	Web Application Firewall with regex rules (ModSecurity) at network layer
5. Query Rewriting	Strip or encode dangerous tokens: replace <code>""</code> with escaped version

Limits of Regex-Only Approach

- ▶ Obfuscation bypass: `'OR' || '1'='1'` or HEX-encoded payloads defeat static patterns
- ▶ False positives: Legitimate queries with apostrophes (O'Brien) flagged incorrectly
- ▶ Latency: Many regex checks per query add measurable overhead at scale
- ▶ Evolving attacks: New injection variants require constant pattern updates

Conclusion

Regex is a useful first layer for SQL injection detection but **MUST** be paired with parameterised queries, input whitelisting, and a WAF. It complements — never replaces — structural defenses.